

Core CS - DBMS



Core CS Interview Prep — DBMS



Structure

Subject	Top 10	Top 25	Top 50
DBMS	Most asked, high-priority	Covers core concepts	Deep dive

DBMS:

→ **Top 10:**

1. What is the difference between DBMS and RDBMS?

A **DBMS** (Database Management System) stores data as files without enforcing relationships between them. An **RDBMS** (Relational DBMS) stores data in structured tables with rows and columns, enforces relationships via foreign keys, and supports ACID properties. Examples: MySQL, PostgreSQL (RDBMS) vs. early file-based systems (DBMS).

2. Explain the ACID properties of a transaction.

- **Atomicity** – All operations in a transaction succeed or none do (all-or-nothing).
- **Consistency** – A transaction brings the database from one valid state to another, maintaining all rules and constraints.
- **Isolation** – Concurrent transactions execute as if they were serial; intermediate states are invisible to others.
- **Durability** – Once a transaction is committed, changes persist even after a system crash (ensured via write-ahead logging).

3. What are the different types of joins in SQL?

- **INNER JOIN** – Returns rows with matching values in both tables.
- **LEFT JOIN** – All rows from the left table + matching rows from the right (NULLs for no match).

- **RIGHT JOIN** – All rows from the right table + matching rows from the left.
 - **FULL OUTER JOIN** – All rows from both tables, NULLs where there's no match.
 - **CROSS JOIN** – Cartesian product of both tables (every row with every row).
-

4. What is normalization? Explain 1NF, 2NF, and 3NF.

Normalization eliminates data redundancy and anomalies by organizing tables using functional dependencies.

- **1NF** – Each column holds atomic (indivisible) values; no repeating groups.
 - **2NF** – Satisfies 1NF + every non-key attribute is fully functionally dependent on the entire primary key (no partial dependencies).
 - **3NF** – Satisfies 2NF + no transitive dependencies (non-key attribute depending on another non-key attribute).
-

5. What is a transaction? What are the different transaction states?

A transaction is a logical unit of work that reads or writes data. States:

- **Active** → currently executing
 - **Partially Committed** → final operation executed, not yet committed
 - **Committed** → changes permanently saved
 - **Failed** → error occurred during execution
 - **Aborted** → rolled back, database restored to prior state
-

6. What is indexing? What are the types and how do they improve performance?

An index is a data structure that speeds up data retrieval without scanning every row.

- **Clustered Index** – Physically sorts the table data; only one per table.
- **Non-Clustered Index** – Separate structure with pointers to actual rows; multiple allowed.
- **B-Tree Index** – Default in most RDBMS; efficient for range queries.
- **Hash Index** – Ideal for exact-match lookups, not range queries.

Indexes reduce query time from $O(n)$ full scans to $O(\log n)$ lookups but add overhead on writes.

7. What is a deadlock? How is it detected and prevented?

A **deadlock** occurs when two or more transactions wait on each other to release locks, causing a circular wait (none can proceed).

Detection: Wait-for graph — if a cycle exists, a deadlock is present. The DBMS typically kills one transaction (victim) to break the cycle.

Prevention strategies:

- **Wait-Die / Wound-Wait** schemes based on transaction timestamps
- **Lock ordering** – always acquire locks in a fixed order
- **Timeouts** – abort transactions that wait too long

8. What is the difference between DELETE, TRUNCATE, and DROP?

Command	Type	Rollback	Where Clause	Removes Structure
DELETE	DML	Yes	Yes	No
TRUNCATE	DDL	No (mostly)	No	No
DROP	DDL	No	No	Yes

- **DELETE** removes specific rows and logs each deletion.
- **TRUNCATE** removes all rows quickly by deallocating data pages.
- **DROP** removes the entire table including its schema.

9. What are triggers and stored procedures? How are they different?

- **Stored Procedure** – A precompiled set of SQL statements executed explicitly by a user or application. Can accept parameters and return values.
- **Trigger** – A special procedure that executes *automatically* in response to an event (INSERT, UPDATE, DELETE) on a table.

Key difference: stored procedures are called manually; triggers fire automatically based on data events.

10. What is the difference between a primary key, unique key, and foreign key?

- **Primary Key** – Uniquely identifies each row; cannot be NULL; only one per table.
- **Unique Key** – Ensures all values in a column are distinct; allows one NULL; multiple per table.
- **Foreign Key** – A column that references the primary key of another table, enforcing **referential integrity** (prevents orphan records).

→ **Top 25:**



DBMS — Top 25 Interview Questions

1. What is a DBMS, and how is it different from a file system?

A DBMS (Database Management System) is software that manages, stores, and retrieves structured data. Unlike a file system, a DBMS provides data abstraction, concurrent access, ACID-compliant transactions, security controls, and eliminates data redundancy through normalization.

2. What are the different types of DBMS?

- **Hierarchical** – data organized in a tree structure (e.g., IBM IMS)
- **Network** – data organized as a graph with many-to-many relationships
- **Relational (RDBMS)** – data stored in tables with relationships (e.g., MySQL, PostgreSQL)
- **Object-Oriented** – data stored as objects (e.g., db4o)
- **NoSQL** – non-relational, schema-less (e.g., MongoDB, Cassandra)

3. What are the ACID properties?

- **Atomicity** – a transaction is all-or-nothing
 - **Consistency** – a transaction brings the DB from one valid state to another
 - **Isolation** – concurrent transactions don't interfere with each other
 - **Durability** – committed transactions persist even after system failure
-

4. What is normalization? Explain 1NF, 2NF, and 3NF.

Normalization is the process of organizing data to reduce redundancy and improve integrity.

- **1NF** – each column holds atomic values; no repeating groups
 - **2NF** – 1NF + no partial dependency (non-key attributes depend on the entire primary key)
 - **3NF** – 2NF + no transitive dependency (non-key attributes depend only on the primary key)
-

5. What is denormalization, and when would you use it?

Denormalization intentionally introduces redundancy to improve read performance. It's used in read-heavy systems (e.g., data warehouses, analytics dashboards) where join cost is high and data consistency can be managed at the application layer.

6. What is a primary key vs. a foreign key?

- **Primary key** – uniquely identifies each row in a table; cannot be NULL
 - **Foreign key** – a column (or set of columns) in one table that references the primary key of another table, enforcing referential integrity
-

7. What is the difference between **INNER JOIN**, **LEFT JOIN**, **RIGHT JOIN**, and **FULL OUTER JOIN**?

- **INNER JOIN** – returns rows with matching values in both tables
 - **LEFT JOIN** – all rows from the left table + matched rows from the right (NULLs for no match)
 - **RIGHT JOIN** – all rows from the right table + matched rows from the left
 - **FULL OUTER JOIN** – all rows from both tables; NULLs where there's no match on either side
-

8. What is an index? What are its types?

An index is a data structure that speeds up data retrieval. Types:

- **Clustered** – physically reorders table data; one per table
- **Non-clustered** – separate structure pointing to data rows; multiple allowed
- **Composite** – index on multiple columns

- **Unique** – enforces uniqueness on indexed columns
 - **Full-text** – optimized for text search
-

9. What is a transaction? What are the different transaction states?

A transaction is a logical unit of work. States:

- **Active** → **Partially Committed** → **Committed**
 - **Active** → **Failed** → **Aborted** (rolled back)
-

10. What is a deadlock? How can it be prevented or resolved?

A deadlock occurs when two or more transactions wait on each other to release locks, creating a cycle. Prevention strategies:

- **Wait-Die / Wound-Wait** schemes
 - **Lock ordering** – always acquire locks in a fixed order
 - **Timeouts** – abort a transaction if it waits too long
 - **Deadlock detection** – use a wait-for graph; abort one transaction to break the cycle
-

11. What is the difference between **DELETE**, **TRUNCATE**, and **DROP**?

- **DELETE** – removes specific rows; can be rolled back; triggers fire; slower
 - **TRUNCATE** – removes all rows; cannot be rolled back (in most RDBMS); faster; resets identity
 - **DROP** – removes the entire table structure and data permanently
-

12. What are aggregate functions? Give examples.

Aggregate functions perform calculations on a set of rows and return a single value:

COUNT(), **SUM()**, **AVG()**, **MIN()**, **MAX()**

They are used with **GROUP BY** and can be filtered using **HAVING**.

13. What is the difference between **WHERE** and **HAVING**?

- **WHERE** filters rows *before* grouping
 - **HAVING** filters groups *after* **GROUP BY** has been applied
-

14. What is a view? What are its advantages?

A view is a virtual table based on a SELECT query. Advantages:

- Simplifies complex queries
 - Provides an abstraction layer / security (hides sensitive columns)
 - Does not store data (unless materialized)
-

15. What is a materialized view?

A materialized view stores the result of a query physically on disk and refreshes it periodically. It improves performance for expensive queries but may return slightly stale data, unlike a regular view which is always real-time.

16. What is a stored procedure vs. a function?

- **Stored procedure** – precompiled SQL block; can have IN/OUT parameters; may or may not return a value; can perform DML operations
 - **Function** – must return a single value; can be used inside a SELECT statement; generally no side effects (no DML in most RDBMS)
-

17. What is a trigger?

A trigger is a stored procedure that automatically executes in response to a DML event (`INSERT` , `UPDATE` , `DELETE`) on a table. Types: `BEFORE` / `AFTER` and `FOR EACH ROW` / `FOR EACH STATEMENT` .

18. What are different types of keys in a DBMS?

- **Super key** – any set of attributes that uniquely identifies a row
 - **Candidate key** – minimal super key
 - **Primary key** – chosen candidate key
 - **Alternate key** – candidate keys not chosen as primary
 - **Foreign key** – references primary key of another table
 - **Composite key** – primary key made of multiple columns
 - **Surrogate key** – system-generated key (e.g., auto-increment ID)
-

19. What is the difference between SQL and NoSQL databases?

Aspect	SQL	NoSQL
Schema	Fixed, predefined	Flexible / schema-less
Scaling	Vertical	Horizontal
Transactions	ACID	BASE (eventual consistency)
Use case	Structured, relational data	Unstructured, high-volume data
Examples	MySQL, PostgreSQL	MongoDB, Cassandra, Redis

20. What is the CAP theorem?

In a distributed system, you can guarantee only **two** of three properties:

- **Consistency** – every read returns the most recent write
- **Availability** – every request gets a response (not necessarily the latest data)
- **Partition Tolerance** – system continues despite network partitions

Since network partitions are unavoidable, real systems choose between CP (e.g., HBase) or AP (e.g., Cassandra).

21. What is database sharding?

Sharding is horizontal partitioning — splitting a large database into smaller pieces (shards) distributed across multiple servers. Each shard holds a subset of the data. It improves scalability and throughput but adds complexity to queries that span multiple shards.

22. What is the difference between optimistic and pessimistic locking?

- **Pessimistic locking** – locks a record before reading/updating; prevents conflicts but reduces concurrency
- **Optimistic locking** – reads data without locking; checks at commit time if another transaction modified it (using a version number or timestamp); rolls back if conflict detected

23. What are the different levels of transaction isolation?

Listed from lowest to highest isolation (and fewest to most restrictions):

1. **Read Uncommitted** – dirty reads possible
2. **Read Committed** – prevents dirty reads; non-repeatable reads possible

3. **Repeatable Read** – prevents dirty + non-repeatable reads; phantom reads possible
 4. **Serializable** – full isolation; prevents all anomalies; lowest concurrency
-

24. What is an ER diagram? What are its components?

An Entity-Relationship (ER) diagram is a visual representation of the database schema. Components:

- **Entity** – a real-world object (rectangle)
 - **Attribute** – property of an entity (ellipse); can be key, composite, derived, or multivalued
 - **Relationship** – association between entities (diamond)
 - **Cardinality** – defines 1:1, 1:N, M:N relationships
-

25. What is query optimization? How does a query optimizer work?

Query optimization is the process of selecting the most efficient execution plan for a SQL query. The optimizer:

1. **Parses** the query into a parse tree
2. **Generates** multiple equivalent logical plans
3. **Estimates cost** of each plan using statistics (row counts, index availability, I/O cost)
4. **Selects** the plan with the lowest estimated cost

Key techniques include index selection, join reordering, predicate pushdown, and avoiding full table scans.

→ **Top 50:**



DBMS — Top 50 Interview Questions & Answers

→ **Q1: What is a DBMS?**

A Database Management System (DBMS) is software that manages the creation, storage, retrieval, and manipulation of data in a database. It provides an interface between the user/application and the physical database, ensuring data integrity, security, and consistency.

→ **Q2: What is the difference between DBMS and RDBMS?**

A DBMS stores data as files without enforcing relationships between them. An RDBMS (Relational DBMS) stores data in tables with defined relationships, enforces referential integrity, and supports SQL. Examples: MySQL, PostgreSQL, Oracle are RDBMS; older systems like dBASE were plain DBMS.

→ **Q3: What are the different types of database models?**

- **Hierarchical** – data in tree structure (parent-child)
 - **Network** – data as graph with multiple parent-child links
 - **Relational** – data in tables with relationships
 - **Object-oriented** – data as objects
 - **NoSQL** – document, key-value, columnar, graph stores
-

→ **Q4: What is a schema?**

A schema is the logical structure of a database — it defines how data is organized, including tables, columns, data types, constraints, and relationships. It does not contain the actual data, only its blueprint.

→ **Q5: What are the three levels of data abstraction in DBMS?**

1. **Physical level** – how data is physically stored on disk
 2. **Logical level** – what data is stored and the relationships (schema)
 3. **View level** – how different users see a subset of the data
-

→ **Q6: What is a primary key?**

A primary key is a column (or combination of columns) that uniquely identifies each row in a table. It cannot be NULL and must be unique across all rows.

→ **Q7: What is a foreign key?**

A foreign key is a column in one table that references the primary key of another table. It enforces referential integrity — ensuring that a value in the foreign key column must exist in the referenced table.

→ **Q8: What is normalization? Why is it important?**

Normalization is the process of organizing a database to reduce redundancy and improve data integrity. It breaks large tables into smaller, related ones. It's

important because it eliminates update anomalies, insert anomalies, and delete anomalies.

→ **Q9: Explain 1NF, 2NF, 3NF, and BCNF.**

- **1NF** – Each column has atomic (indivisible) values; no repeating groups.
 - **2NF** – 1NF + every non-key attribute is fully dependent on the entire primary key (removes partial dependencies).
 - **3NF** – 2NF + no transitive dependencies (non-key attributes don't depend on other non-key attributes).
 - **BCNF** – Stricter version of 3NF; for every functional dependency $X \rightarrow Y$, X must be a superkey.
-

→ **Q10: What is denormalization?**

Denormalization is intentionally introducing redundancy into a normalized database to improve read performance. It reduces the number of joins needed for queries, at the cost of increased storage and potential update anomalies. Common in data warehouses and read-heavy systems.

→ **Q11: What is a transaction in DBMS?**

A transaction is a sequence of one or more database operations (read/write) treated as a single logical unit. It either completes fully (commit) or is rolled back entirely (rollback) if any part fails.

→ **Q12: What are the ACID properties?**

- **Atomicity** – All operations in a transaction succeed or none do.
 - **Consistency** – A transaction brings the database from one valid state to another.
 - **Isolation** – Concurrent transactions execute as if they were serial.
 - **Durability** – Once committed, changes persist even after system failures.
-

→ **Q13: What is the difference between DELETE, TRUNCATE, and DROP?**

- **DELETE** – Removes specific rows; can be rolled back; triggers fire; logs each row deletion.
- **TRUNCATE** – Removes all rows from a table; faster; cannot be rolled back in most DBs; resets auto-increment.

- **DROP** – Removes the entire table structure and its data permanently.
-

→ **Q14: What is an index? What are its types?**

An index is a data structure (typically a B-tree or hash) that speeds up data retrieval. Types include:

- **Clustered** – determines physical order of rows in a table; one per table
 - **Non-clustered** – separate structure pointing to rows; multiple allowed
 - **Composite** – index on multiple columns
 - **Unique** – ensures no duplicate values
 - **Full-text** – for text search
-

→ **Q15: What is the difference between clustered and non-clustered indexes?**

A clustered index sorts and stores the actual data rows in the table based on the index key — there can only be one. A non-clustered index is a separate structure that stores the index key and a pointer to the actual data row — multiple non-clustered indexes can exist per table.

→ **Q16: What is a view? How is it different from a table?**

A view is a virtual table defined by a SQL query. It doesn't store data itself — it pulls data dynamically from the underlying tables when queried. Unlike tables, views don't physically store data (except materialized views). Views are used for security, simplification, and abstraction.

→ **Q17: What is a stored procedure?**

A stored procedure is a precompiled collection of SQL statements stored in the database and executed as a unit. It accepts input/output parameters, reduces network traffic, improves performance through precompilation, and encapsulates business logic on the database side.

→ **Q18: What is a trigger?**

A trigger is a stored procedure that automatically executes in response to a specific event (INSERT, UPDATE, DELETE) on a table or view. Triggers are used to enforce business rules, maintain audit logs, or cascade changes.

→ **Q19: What is a join? Explain its types.**

A join combines rows from two or more tables based on a related column.

- **INNER JOIN** – returns rows with matching values in both tables
 - **LEFT JOIN** – all rows from left table + matching rows from right
 - **RIGHT JOIN** – all rows from right table + matching from left
 - **FULL OUTER JOIN** – all rows from both tables; NULLs where no match
 - **CROSS JOIN** – Cartesian product of both tables
 - **SELF JOIN** – a table joined with itself
-

→ **Q20: What is the difference between UNION and UNION ALL?**

Both combine results of two SELECT queries. **UNION** removes duplicate rows; **UNION ALL** includes all rows including duplicates. UNION ALL is faster since it skips the deduplication step.

→ **Q21: What is a subquery? What are correlated subqueries?**

A subquery is a query nested inside another query. A **correlated subquery** references columns from the outer query and is re-executed for every row of the outer query, making it row-dependent and often slower than regular subqueries.

→ **Q22: What are aggregate functions in SQL?**

Aggregate functions perform calculations on a set of rows and return a single value:

- `COUNT()` – number of rows
- `SUM()` – total of a numeric column
- `AVG()` – average value
- `MIN()` / `MAX()` – minimum/maximum value

They are used with `GROUP BY` to compute values per group.

→ **Q23: What is the difference between WHERE and HAVING?**

WHERE filters rows before grouping — it works on individual rows and cannot use aggregate functions. **HAVING** filters groups after `GROUP BY` — it works on aggregated results and can use aggregate functions.

→ **Q24: What is a deadlock? How can it be prevented?**

A deadlock occurs when two or more transactions each hold a lock and wait for the other to release it, forming a circular dependency. Prevention strategies include:

- Lock ordering (always acquire locks in the same order)
 - Using timeouts
 - Deadlock detection and rollback
 - Using optimistic concurrency control
-

→ **Q25: What is concurrency control?**

Concurrency control ensures that multiple transactions can execute simultaneously without causing data inconsistency. Techniques include **locking** (shared/exclusive locks), **timestamp ordering**, and **multiversion concurrency control (MVCC)**.

→ **Q26: What are the different types of database keys?**

- **Primary key** – uniquely identifies a row
 - **Foreign key** – references another table's primary key
 - **Candidate key** – any column(s) that could serve as a primary key
 - **Super key** – any set of columns that uniquely identifies a row (superset of candidate key)
 - **Composite key** – primary key made of multiple columns
 - **Alternate key** – candidate keys not chosen as primary key
 - **Surrogate key** – system-generated artificial key (e.g., auto-increment ID)
-

→ **Q27: What is referential integrity?**

Referential integrity is a constraint that ensures that a foreign key value in one table always corresponds to an existing primary key value in the referenced table. It prevents orphaned records and maintains consistency between related tables.

→ **Q28: What is a cursor in DBMS?**

A cursor is a database object used to retrieve, manipulate, and navigate through a result set row by row. It's used in stored procedures when set-based

operations aren't sufficient. Types include **implicit** (auto-created by DB engine) and **explicit** (declared manually by developer).

→ **Q29: What is the difference between optimistic and pessimistic locking?**

- **Pessimistic locking** – locks a record when it's read, preventing others from modifying it until the lock is released. Suitable for high-conflict environments.
 - **Optimistic locking** – doesn't lock on read; checks at update time whether data has changed (using a version/timestamp). Suitable for low-conflict environments.
-

→ **Q30: What is a materialized view?**

A materialized view physically stores the query result on disk, unlike a regular view which is computed on the fly. It improves performance for expensive queries but needs to be refreshed periodically to stay in sync with the base tables.

→ **Q31: What is partitioning in databases?**

Partitioning divides a large table into smaller, more manageable pieces while still being accessed as a single table. Types:

- **Horizontal (sharding)** – rows split across partitions
 - **Vertical** – columns split across partitions
 - **Range, List, Hash** – common partition strategies based on key values
-

→ **Q32: What is sharding?**

Sharding is a type of horizontal partitioning where data is distributed across multiple database servers (shards). Each shard holds a subset of the data. It improves scalability and performance for very large datasets but adds complexity in querying and maintaining consistency.

→ **Q33: What is the CAP theorem?**

The CAP theorem states that a distributed database can guarantee at most **two** of three properties simultaneously:

- **Consistency** – every read receives the most recent write
- **Availability** – every request receives a response

- **Partition tolerance** – system continues despite network partitions

In practice, partition tolerance is necessary, so systems trade off between C and A.

→ **Q34: What is the difference between SQL and NoSQL databases?**

	SQL	NoSQL
Structure	Tables with fixed schema	Flexible schema (document, key-value, etc.)
Scaling	Vertical	Horizontal
ACID	Fully supported	Eventually consistent (usually)
Use case	Structured, relational data	Unstructured, large-scale, fast-changing data

→ **Q35: What is database replication?**

Replication is the process of copying and maintaining database data across multiple servers. It improves availability and fault tolerance. Types include **master-slave** (one write source, multiple read replicas) and **master-master** (multiple nodes can write).

→ **Q36: What is a data warehouse?**

A data warehouse is a large, centralized repository of integrated data from multiple sources, optimized for analytical queries and reporting rather than transactional operations. It uses schemas like **star schema** or **snowflake schema** and is the foundation for BI and analytics.

→ **Q37: What is the difference between OLTP and OLAP?**

- **OLTP (Online Transaction Processing)** – handles high-volume, short, real-time transactions (INSERT/UPDATE/DELETE). Highly normalized. Used in operational systems.
- **OLAP (Online Analytical Processing)** – handles complex analytical queries on large historical datasets. Denormalized. Used in data warehouses and reporting.

→ **Q38: What are functional dependencies?**

A functional dependency $X \rightarrow Y$ means that the value of X uniquely determines the value of Y. For example, if `StudentID → StudentName`, then knowing the student

ID uniquely identifies the student's name. Functional dependencies are the foundation of normalization theory.

→ **Q39: What is the difference between a natural join and an equi-join?**

An **equi-join** uses an explicit equality condition (`ON A.id = B.id`) and may retain duplicate columns. A **natural join** automatically joins on all columns with the same name in both tables and removes duplicate columns. Natural join is implicit; equi-join is explicit.

→ **Q40: What is a composite key? When would you use it?**

A composite key is a primary key made up of two or more columns whose combined values uniquely identify a row. It's used when no single column is unique enough on its own — for example, in a junction table like

```
StudentCourse(StudentID, CourseID) .
```

→ **Q41: What are savepoints in transactions?**

A savepoint is a named marker within a transaction that allows partial rollback. Instead of rolling back the entire transaction, you can roll back to the savepoint and continue from there. Useful in long transactions where only a portion may fail.

→ **Q42: What is the difference between a hot backup and a cold backup?**

- **Cold backup** – taken when the database is offline; data files are consistent but the database is unavailable during backup.
 - **Hot backup** – taken while the database is running; uses archive logs to ensure consistency. No downtime but more complex.
-

→ **Q43: What is query optimization?**

Query optimization is the process by which the DBMS query optimizer selects the most efficient execution plan for a SQL query. It considers factors like available indexes, table statistics, join order, and access methods to minimize cost (I/O, CPU, memory).

→ **Q44: What is an execution plan?**

An execution plan (or explain plan) is the step-by-step strategy chosen by the query optimizer to execute a SQL statement. It shows operations like table scans, index seeks, join methods, and sort steps. Analyzing it helps identify bottlenecks and optimize slow queries.

→ **Q45: What is the N+1 query problem?**

The N+1 problem occurs when an application executes 1 query to fetch a list of N records, then N additional queries to fetch related data for each record individually. This causes massive performance issues. It's resolved using **JOIN**s or eager loading in ORMs.

→ **Q46: What is database isolation and what are its levels?**

Isolation determines how visible uncommitted changes of one transaction are to others. Levels (from lowest to highest isolation):

1. **Read Uncommitted** – can read dirty data
 2. **Read Committed** – only reads committed data
 3. **Repeatable Read** – same rows return same data throughout the transaction
 4. **Serializable** – transactions are fully isolated; no phantom reads
-

→ **Q47: What are dirty reads, phantom reads, and non-repeatable reads?**

- **Dirty read** – reading uncommitted data from another transaction that may later be rolled back
 - **Non-repeatable read** – re-reading a row within the same transaction returns different values because another transaction updated it
 - **Phantom read** – re-running a query returns different rows because another transaction inserted or deleted rows
-

→ **Q48: What is the difference between a sequence and an identity column?**

Both generate auto-incrementing numbers. An **identity column** is a table property — the number is automatically assigned on INSERT and is tied to the table. A **sequence** is a standalone database object that generates numbers independently and can be shared across multiple tables.

→ **Q49: What is connection pooling?**

Connection pooling is a technique where a pool of pre-established database connections is maintained and reused across requests, rather than opening and closing a new connection for each operation. It significantly reduces connection overhead and improves application throughput.

→ **Q50: What is the difference between vertical and horizontal scaling in databases?**

- **Vertical scaling (scale up)** – adding more CPU, RAM, or storage to a single server. Simple but has hardware limits.
- **Horizontal scaling (scale out)** – adding more servers/nodes to distribute load. Achieved through replication, sharding, or clustering. More complex but theoretically limitless.